3.2 Loading Mechanism

Rails autoloads files with `Kernel#load` when `config.cache_classes` is false, the default in development mode, and with `Kernel#require` otherwise, the default in production mode.
`Kernel#load` allows Rails to execute files more than once if constant reloading is enabled.
This guide uses the word "load" freely to mean a given file is interpreted, but the actual mechanism can be `Kernel#load` or `Kernel#require` depending on that flag.

4 Autoloading Availability

Rails is always able to autoload provided its environment is in place. For example the `runner`command autoloads:

```
$ bin/rails runner 'p User.column_names'
["id", "email", "created_at", "updated_at"]
```

The console autoloads, the test suite autoloads, and of course the application autoloads.

By default, Rails eager loads the application files when it boots in production mode, so most of the autoloading going on in development does not happen. But autoloading may still be triggered during eager loading.

For example, given

```
class BeachHouse < House
end
```

if `House` is still unknown when `app/models/beach_house.rb` is being eager loaded, Rails autoloads it.

5 autoload_paths

As you probably know, when `require` gets a relative file name:

```
require 'erb'
```

Ruby looks for the file in the directories listed in `$LOAD_PATH`. That is, Ruby iterates over all its directories and for each one of them checks whether they have a file called "erb.rb", or "erb.so", or "erb.o", or "erb.dll". If it finds any of them, the interpreter loads it and ends the search. Otherwise, it tries again in the next directory of the list. If the list gets exhausted, `LoadError` is raised.
We are going to cover how constant autoloading works in more detail later, but the idea is that when a constant like `Post` is hit and missing, if there's a `post.rb` file for example in `app/models` Rails is going to find it, evaluate it, and have `Post` defined as a side-effect.
Alright, Rails has a collection of directories similar to `$LOAD_PATH` in which to look up `post.rb`. That collection is called `autoload_paths` and by default it contains:

- All subdirectories of `app` in the application and engines. For example, `app/controllers`. They do not need to be the default ones, any custom directories like `app/workers` belong automatically to `autoload_paths`.
- Any existing second level directories called `app/*/concerns` in the application and engines.
- The directory `test/mailers/previews`.

Also, this collection is configurable via `config.autoload_paths`. For example, `lib` was in the list years ago, but no longer is. An application can opt-in by adding this to `config/application.rb`:

```
config.autoload_paths << "#{Rails.root}/lib"
```

`config.autoload_paths` is accessible from environment-specific configuration files, but any changes made to it outside `config/application.rb` don't have an effect.
The value of `autoload_paths` can be inspected. In a just generated application it is (edited):

```
$ bin/rails r 'puts ActiveSupport::Dependencies.autoload_paths'
.../app/assets
```